UNIT 4.Classification

Classification is a core task in Data Mining and Machine Learning where the goal is to assign data into predefined categories (classes).

- It is a **supervised learning** technique
- Uses **labeled training data** to learn a model
- The model predicts class labels for new (unseen) data

Supervised Learning

Supervised learning is a method where the model is trained using **labeled data**.

Example

Input (Features)	Output (Label)
Email text	Spam / Not Spam
Patient data	Disease / No Disease

Common Algorithms

- Decision Tree
- Naive Bayes classifier
- Support Vector Machine

Unsupervised Learning

Unsupervised learning works with **unlabeled data**.

Example

- Group customers based on buying behavior
- Detect unusual transactions (fraud detection)

Common Algorithms

- K-means clustering
- Hierarchical clustering
- DBSCAN

ID3

The **ID3 algorithm (Iterative Dichotomiser 3)** is a classic method used in **Machine Learning** to build a **decision tree** for classification tasks.

Core Idea

ID3 builds a tree **top-down** by repeatedly:

1. Selecting the **best attribute (feature)** to split the dataset.
2. Splitting the data based on that attribute.
3. Recursively repeating the process for each subset.

Key Concept: Information Gain

The "best" feature is chosen using **Information Gain**, which is based on **Entropy**.

1. Entropy

Entropy tells how mixed the dataset is:

- If all examples belong to one class → entropy = 0 (pure)
- If evenly mixed → entropy is high

Formula:

$$Entropy(S) = -\sum p_i \log_2 p_i$$

---

2. Information Gain

It measures how much uncertainty is reduced after splitting on an attribute.

$$Gain(S, A) = Entropy(S) - \sum \frac{|S_v|}{|S|} Entropy(S_v)$$

- S: dataset
- A: attribute
- Sv: subsets after splitting

Dataset (Play Tennis Example)

Day	Outlook	Humidity	Wind	Play
1	Sunny	High	Weak	No
2	Sunny	High	Strong	No
3	Overcast	High	Weak	Yes
4	Rain	High	Weak	Yes
5	Rain	Normal	Weak	Yes
6	Rain	Normal	Strong	No
7	Overcast	Normal	Strong	Yes
8	Sunny	High	Weak	No

Step 1: Entropy of Dataset

Number of Yes = 4

Number of No = 4

Formula:

Entropy(S) = − p(Yes) $\log_2$ p(Yes) − p(No) $\log_2$ p(No)

Calculation:

Entropy(S) = − (4/8 $\log_2$ 4/8) − (4/8 $\log_2$ 4/8)

Entropy(S) = 1

Step 2: Information Gain Calculation

Feature: Outlook

Split Data:

- Sunny → No, No, No → Entropy = 0
- Overcast → Yes, Yes → Entropy = 0
- Rain → Yes, Yes, No

Entropy(Rain) =

− (2/3 $\log_2$ 2/3) − (1/3 $\log_2$ 1/3) =0.918

Weighted Entropy:

= (3/8 × 0) + (2/8 × 0) + (3/8 × 0.918)

= 0.344

Information Gain:

Gain(Outlook) = Entropy(S) – Weighted Entropy

= 1 − 0.344 = **0.656**

Feature: Humidity

- High → 2 Yes, 3 No → Entropy ≈ 0.971
- Normal → 2 Yes, 1 No → Entropy ≈ 0.918

Entropy(High) and Entropy(Normal) should be calculated as above.

Weighted Entropy:

= (5/8 × 0.971) + (3/8 × 0.918)

= 0.95

Gain(Humidity) = 1 − 0.95 = **0.05**

Feature: Wind

- Weak → 3 Yes, 2 No → Entropy ≈ 0.971
- Strong → 1 Yes, 2 No → Entropy ≈ 0.918

Entropy(Weak) and Entropy(Strong) should be calculated as above.

Weighted Entropy

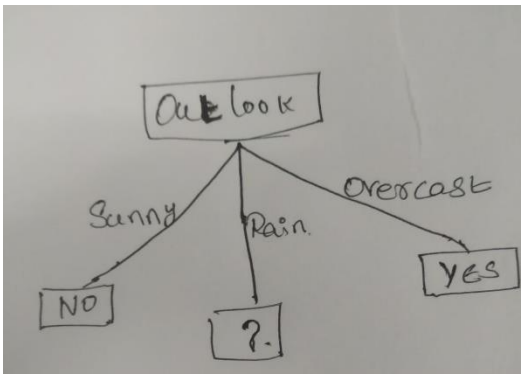
= 0.95

Gain(Wind) = 1 − 0.95 = **0.05**

Step 3: Root Node Selection

Feature	Information Gain
Outlook	**0.656 (Highest)**
Humidity	0.05
Wind	0.05

Root Node = **Outlook**



Step 4: Recursive Tree Construction

In the above tree

Outlook = Sunny

All values = No

Leaf Node = **No**

Hence No Further Branch

Outlook = Overcast

All values = Yes

Leaf Node = **Yes**

Hence No Further Branch

Outlook = Rain

Subset:

Humidity	Wind	Play
High	Weak	Yes
Normal	Weak	Yes
Normal	Strong	No

Note: The above data set is reduced data set from original. In this the best feature (outlook) and only rows (outlook = Rain) is considered

Choose Best Feature (Based on the above dataset)

Same procedure should be used for calculation.

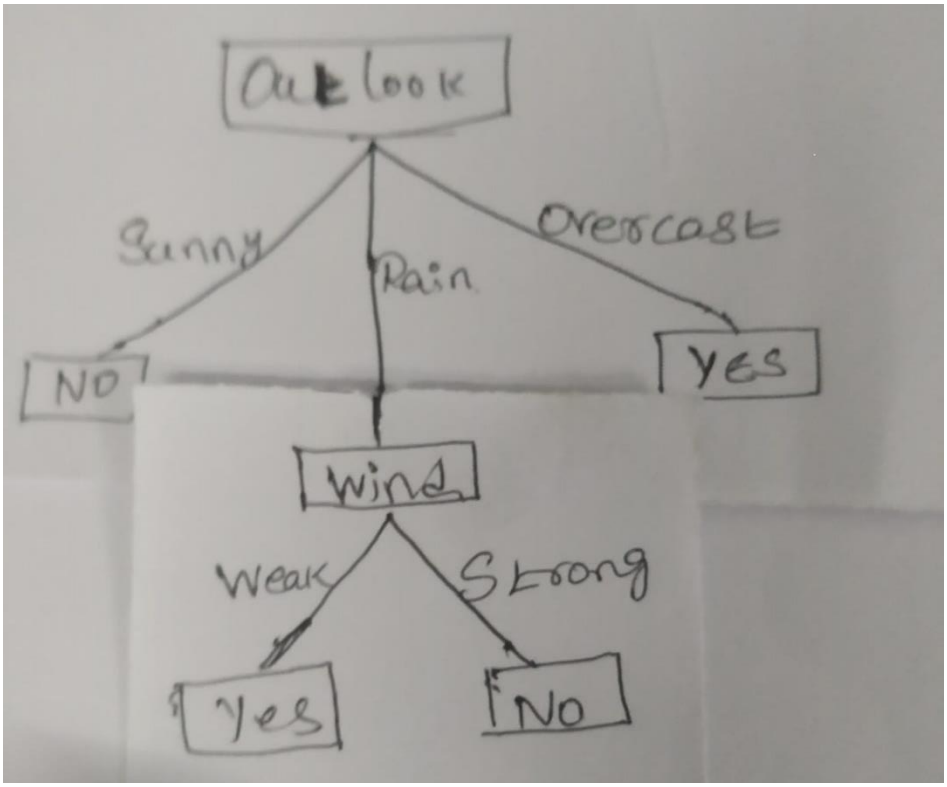
Best Feature(Wind)

- Weak → Yes, Yes → Entropy = 0
- Strong → No → Entropy = 0

Best split = **Wind**

---

7. Final Decision Tree



8. Final Rules

1. If Outlook = Sunny → No
2. If Outlook = Overcast → Yes
3. If Outlook = Rain AND Wind = Weak → Yes
4. If Outlook = Rain AND Wind = Strong → No

NAIVE BAYES ALGORITHM

Naïve Bayes predicts the probability that a given data tuple belongs to a particular class.

It is based on **Bayes' Theorem**:

$$P(C \mid X) = \frac{P(X \mid C)P(C)}{P(X)}$$

$$P(A)$$

$$P(B \mid A)$$

$$P(B \mid \neg A)$$

$$P(A \mid B) = \frac{P(B \mid A)P(A)}{P(B)} \approx 0.68,\ P(B) \approx 0.25$$

P(B)=0.25P(B|A)P(A)=0.17P(A|B)~0.68Posterior = useful evidence / total evidence

Where:

- **P(C|X)** = Posterior probability → probability of class **C** given predictor **X**
- **P(X|C)** = Likelihood → probability of predictor **X** given class **C**
- **P(C)** = Prior probability of class
- **P(X)** = Prior probability of predictor

---

2. Why is it called "Naive"?

It is called **naive** because it assumes that:

All attributes are independent of each other, given the class label.

For example, if we classify whether a customer buys a computer based on:

- Age
- Income
- Student status

Naïve Bayes assumes these factors are independent, which is often not true in real life—but the algorithm still performs surprisingly well.

---

3. Naive Bayes in Data Mining

In **data mining**, Naive Bayes is used for:

- **Customer purchase prediction**
- **Spam email detection**
- **Document classification**
- **Medical diagnosis**
- **Fraud detection**

It works well in **data warehouses** because:

- It handles **large datasets efficiently**
- It is **fast to train**
- It requires **less training data**
- It works well with **categorical data**

---

4. Example: Customer Buying Prediction

Suppose a company wants to predict whether a customer will buy a laptop.

We have the following training data:

Age	Income	Student	Buys Laptop
Young	High	No	No
Young	High	No	No
Middle	High	No	Yes
Old	Medium	No	Yes
Old	Low	Yes	Yes
Old	Low	Yes	No
Middle	Low	Yes	Yes
Young	Medium	No	No

Age	Income	Student	Buys Laptop
Young	Low	Yes	Yes
Old	Medium	Yes	Yes

Now classify this new customer:

Age = Young, Income = Medium, Student = Yes

Will they buy a laptop?

---

5. Step 1: Calculate Prior Probabilities

From dataset:

- **Yes = 6**
- **No = 4**

So:

$$P(Yes) = \frac{6}{10} = 0.6, P(No) = \frac{4}{10} = 0.4$$

---

6. Step 2: Conditional Probabilities

For class **Yes**:

- P(Young | Yes) = 1/6
- P(Medium | Yes) = 2/6
- P(Yes student | Yes) = 4/6

For class **No**:

- P(Young | No) = 3/4
- P(Medium | No) = 1/4
- P(Yes student | No) = 1/4

---

7. Step 3: Apply Naïve Bayes Formula

For **Yes**:

$$P(Yes \mid X) \propto P(Yes) \times P(Young \mid Yes) \times P(Medium \mid Yes) \times P(Student = Yes \mid Yes)$$

$$= 0.6 \times \frac{1}{6} \times \frac{2}{6} \times \frac{4}{6}$$
$$= 0.0222$$

For **No**:

$$P(No \mid X) \propto P(No) \times P(Young \mid No) \times P(Medium \mid No) \times P(Student = Yes \mid No)$$

$$= 0.4 \times \frac{3}{4} \times \frac{1}{4} \times \frac{1}{4}$$
$$= 0.01875$$

---

8. Final Decision

Since:

$$0.0222 > 0.01875$$

The predicted class is:

Buys Laptop = Yes

So this customer is likely to **buy the laptop**.

---

9. Advantages in Data Warehousing & Mining

Advantages:

- Simple to implement
- Fast for large databases
- Works well in real-time prediction
- Good for categorical data

- Scalable in data warehouses

Disadvantages:

- Assumes independence among attributes
- Zero probability issue (handled using Laplace smoothing)
- Less accurate if attributes are strongly dependent

---

10. Role in Data Warehousing

In **data warehousing**, Naïve Bayes can be applied on warehouse data for:

- **Customer segmentation**
- **Sales forecasting**
- **Market basket analysis**
- **Risk analysis**
- **Decision support systems**

Example:

A retail data warehouse stores customer history, and Naïve Bayes predicts:

"Will this customer buy a product based on previous behavior?"

This helps businesses in **decision-making**.

K-MEANS ALGORITHM

The **K-Means algorithm** is one of the most popular **clustering algorithms** in **data mining**. It is used to group similar data objects into clusters based on their features.

Unlike **Naïve Bayes**, which is a **classification algorithm**, **K-Means** is an **unsupervised learning algorithm**, meaning:

It does **not require predefined class labels**.

---

1. What is K-Means?

K-Means partitions data into **K clusters**, where:

- **K** = number of clusters
- Each cluster has a **centroid** (center point)
- Data points are assigned to the nearest centroid

The goal is to minimize the distance between data points and their cluster centroid.

---

2. Objective Function of K-Means

K-Means minimizes the **sum of squared distances** between data points and the centroid:

$$J = \sum_{j=1}^{K} \sum_{i=1}^{n} \| x_i^{(j)} - c_j \|^2$$

Where:

- **K** = number of clusters
- $x_i^{(j)}$= data point in cluster **j**
- c_j= centroid of cluster **j**

---

3. Steps of K-Means Algorithm

Step 1:

Choose the number of clusters **K**

Step 2:

Initialize **K centroids** randomly

Step 3:

Assign each data point to the nearest centroid

Step 4:

Recalculate the centroids

Step 5:

Repeat steps 3 and 4 until centroids do not change

---

4. Example of K-Means Clustering

Suppose we have the following 1-dimensional data:

$$2,\ 4,\ 10,\ 12$$

Let:

K = 2

That means we want **2 clusters**.

---

5. Step 1: Select Initial Centroids

Assume initial centroids are:

$$C_1 = 2, C_2 = 10$$

---

6. Step 2: Assign Points to Nearest Centroid

Calculate distances:

Data Point	Distance to C_1=2	Distance to C_2=10	Cluster
2	0	8	C_1
4	2	6	C_1
10	8	0	C_2
12	10	2	C_2

So clusters become:

- **Cluster 1:** {2, 4}
- **Cluster 2:** {10, 12}

---

7. Step 3: Recalculate Centroids

New centroid for Cluster 1:

$$C_1 = \frac{2+4}{2} = 3$$

New centroid for Cluster 2:

$$C_2 = \frac{10+12}{2} = 11$$

Updated centroids:

$$C_1 = 3, C_2 = 11$$

---

8. Step 4: Reassign Data Points

Again calculate distances:

Data Point	Distance to C_1=3	Distance to C_2=11	Cluster
2	1	9	C_1
4	1	7	C_1
10	7	1	C_2
12	9	1	C_2

Clusters remain the same:

- **Cluster 1:** {2,4}
- **Cluster 2:** {10,12}

Centroids do not change, so the algorithm stops.

---

9. Final Clusters

Cluster 1:

$$\{2, 4\}$$

Cluster 2:

$$\{10, 12\}$$

---

10. Applications of K-Means in Data Mining

K-Means is widely used in:

- **Customer segmentation**
- **Market analysis**
- **Image compression**
- **Document clustering**
- **Fraud detection**
- **Pattern recognition**

Example:

A retail company groups customers into clusters based on:

- Purchase amount
- Frequency of shopping
- Product preferences

This helps in **targeted marketing**.

---

11. Advantages of K-Means

- Simple and easy to implement
- Fast for large datasets
- Works well when clusters are clearly separated
- Efficient in data mining tasks

---

12. Disadvantages of K-Means

- Need to specify **K** in advance
- Sensitive to initial centroid selection
- Does not work well with irregular-shaped clusters
- Sensitive to outliers

---

13. K-Means in Data Warehousing

In **data warehousing**, K-Means helps in:

- Grouping customers based on sales data
- Finding purchasing patterns
- Market segmentation
- Business intelligence analysis

Example:

From warehouse customer data, K-Means can identify:

“Which customers belong to high-value, medium-value, and low-value groups?”

This helps management make strategic decisions.